

Enterprise Business Mail Suite

Developer Implementation Plan — Phase-wise & Feature-wise Breakdown with Database Schema

September 2026

Contents

Introduction	1
1. Pre-requisites (Phase 0 — Week 0)	2
2. Full Database Schema (PostgreSQL)	2
2.1 tenants	2
2.2 domains	2
2.3 dns_verification_logs	3
2.4 mailboxes (virtual users)	3
2.5 two_factor_backup_codes	3
2.6 refresh_tokens	4
2.7 aliases	4
2.8 sieve_scripts	4
2.9 caldav_carddav_objects (Radicale-linked)	4
2.10 contacts_cache (auto-suggestion, Redis-backed with Postgres fallback)	5
2.11 migration_jobs	5
2.12 rate_limit_policies	5
2.13 audit_logs	6
2.14 billing_subscriptions	6
2.15 webhooks (outbound event notifications for enterprise clients)	6
2.16 deletion_requests (right-to-erasure compliance)	6
3. Phase-wise and Feature-wise Plan	7
Phase 1 — Foundation (Weeks 1-2)	7
Phase 2 — Management (Weeks 3-4)	8
Phase 3 — Collaboration: Calendar and Contacts (Weeks 5-7)	9
Phase 4 — Security and Hardening (Weeks 8-9)	10
Phase 5 — Scale and Mobile Readiness (Week 10 onward)	11
4. Cross-Cutting Concerns (Ongoing, Not Tied to a Single Phase)	12
5. Suggested Sprint Grouping (Small Team)	12

Introduction

This document translates the system architecture (Postfix, Dovecot, Rspamd, PostgreSQL, Node.js, React, Radicale) into a phase-wise, feature-level implementation plan for the devel-

opment team. Each phase lists infrastructure tasks, backend API endpoints, frontend screens, and the database tables involved, down to the smallest individual feature, so that items can be moved directly into sprint tickets. A complete database schema is included in Section 2.

1. Pre-requisites (Phase 0 — Week 0)

The following must be in place before development begins:

- Server provisioning: minimum two VMs for development (one for the mail engine, one for the application/API layer), Ubuntu 22.04/24.04
- Domain and DNS access for testing MX, SPF, DKIM, and DMARC records
- Docker and Docker Compose installed on the development environment
- PostgreSQL 15+ instance (local Docker container for development)
- Redis instance (for BullMQ job queues and the contact auto-suggestion cache)
- Repository structure decided: mail-engine/ (configs), api/ (Node.js), webmail/ (React), admin/ (React), calendar/ (React) — monorepo or separate repositories, to be agreed upfront
- Node.js LTS version pinned (20.x or 22.x)
- Environment variable and secrets management strategy defined
- CI pipeline skeleton in place (lint, test, build)

2. Full Database Schema (PostgreSQL)

This is the complete schema. In addition to the five tables described in the architecture document, supporting tables have been added that are required in practice but were not explicitly specified (sessions, migration jobs, Sieve scripts, billing, DNS tracking, contacts, audit, deletion requests, etc.). These should be created as migration files at the start of Phase 1.

2.1 tenants

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
name	VARCHAR(255)	NOT NULL	Company / organization name
billing_email	VARCHAR(255)	NOT NULL	
plan	VARCHAR(64)	DEFAULT 'trial'	free / starter / pro / enterprise
storage_region	VARCHAR(64)	DEFAULT 'ap-south-1'	For data-sovereignty compliance
is_active	BOOLEAN	DEFAULT true	
created_at	TIMESTAMPTZ	DEFAULT NOW()	

2.2 domains

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
tenant_id	UUID	FOREIGN KEY → tenants, INDEX	
domain_name	VARCHAR(255)	UNIQUE, NOT NULL	
is_active	BOOLEAN	DEFAULT true	
dns_verified	BOOLEAN	DEFAULT false	
max_users	INTEGER	DEFAULT 50	Billing-tier limit
dkim_private_key	TEXT	ENCRYPTED	
dkim_public_key	TEXT		Shown to admin for DNS setup
dkim_selector	VARCHAR(64)		
spf_verified	BOOLEAN	DEFAULT false	
dmARC_verified	BOOLEAN	DEFAULT false	
mx_verified	BOOLEAN	DEFAULT false	
created_at	TIMESTAMPTZ	DEFAULT NOW()	

2.3 dns_verification_logs

Column	Type	Constraints	Notes
id	BIGSERIAL	PRIMARY KEY	
domain_id	UUID	FOREIGN KEY → domains	
record_type	VARCHAR(16)	NOT NULL	MX / SPF / DKIM / DMARC
status	VARCHAR(16)	NOT NULL	pass / fail / pending
checked_at	TIMESTAMPTZ	DEFAULT NOW()	
raw_response	TEXT		For debugging DNS lookups

2.4 mailboxes (virtual users)

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
domain_id	UUID	FOREIGN KEY → domains	
email	VARCHAR(255)	UNIQUE, NOT NULL	Dovecot login
password_hash	VARCHAR(255)	NOT NULL	ARGON2ID
quota_bytes	BIGINT	DEFAULT 5368709120	
used_bytes	BIGINT	DEFAULT 0	Updated periodically
name	VARCHAR(255)		Display name
role	VARCHAR(32)	DEFAULT 'user'	user / domain_admin / super_admin
two_factor_secret	VARCHAR(64)	NULLABLE	
two_factor_enabled	BOOLEAN	DEFAULT false	
is_active	BOOLEAN	DEFAULT true	
last_login_at	TIMESTAMPTZ		
created_at	TIMESTAMPTZ	DEFAULT NOW()	

2.5 two_factor_backup_codes

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
mailbox_id	UUID	FOREIGN KEY → mailboxes	
code_hash	VARCHAR(255)	NOT NULL	
used	BOOLEAN	DEFAULT false	

2.6 refresh_tokens

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
mailbox_id	UUID	FOREIGN KEY → mailboxes	
token_hash	VARCHAR(255)	NOT NULL	Raw token is never stored
ip_address	INET		
user_agent	TEXT		
expires_at	TIMESTAMPTZ	NOT NULL	
revoked	BOOLEAN	DEFAULT false	
created_at	TIMESTAMPTZ	DEFAULT NOW()	

2.7 aliases

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
domain_id	UUID	FOREIGN KEY → domains	
source	VARCHAR(255)	NOT NULL, INDEX	
destination	VARCHAR(255)	NOT NULL	
is_catchall	BOOLEAN	DEFAULT false	
created_at	TIMESTAMPTZ	DEFAULT NOW()	

2.8 sieve_scripts

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
mailbox_id	UUID	FOREIGN KEY → mailboxes	
name	VARCHAR(128)	NOT NULL	e.g. “active”
raw_script	TEXT	NOT NULL	Compiled Sieve syntax
rule_json	JSONB		Structured version from visual builder
is_active	BOOLEAN	DEFAULT true	
updated_at	TIMESTAMPTZ	DEFAULT NOW()	

2.9 caldav_carddav_objects (Radicale-linked)

Column	Type	Constraints	Notes
collection_id	VARCHAR(255)	NOT NULL, INDEX	Calendar / address-book path
href	VARCHAR(255)	PRIMARY KEY	

Column	Type	Constraints	Notes
data	TEXT	NOT NULL	Raw iCalendar or vCard
etag	VARCHAR(64)	NOT NULL	
last_modified	TIMESTAMPTZ	NOT NULL	

2.10 contacts_cache (auto-suggestion, Redis-backed with Postgres fallback)

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
mailbox_id	UUID	FOREIGN KEY → mailboxes	
email	VARCHAR(255)	NOT NULL	
display_name	VARCHAR(255)		
interaction_count	INTEGER	DEFAULT 1	
last_used_at	TIMESTAMPTZ	DEFAULT NOW()	

2.11 migration_jobs

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
mailbox_id	UUID	FOREIGN KEY → mailboxes	Destination mailbox
source_provider	VARCHAR(64)		gmail / o365 / cpanel / exchange
source_host	VARCHAR(255)		
source_auth_type	VARCHAR(32)		password / oauth2
source_credentials_enc	TEXT		Encrypted
status	VARCHAR(32)	DEFAULT 'queued'	queued / running / paused / done / failed
progress_percent	INTEGER	DEFAULT 0	
last_log_line	TEXT		
started_at	TIMESTAMPTZ		
completed_at	TIMESTAMPTZ		

2.12 rate_limit_policies

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
domain_id	UUID	FOREIGN KEY → domains, NULLABLE	NULL = global default
max_per_hour	INTEGER	DEFAULT 300	
max_per_6h	INTEGER	DEFAULT 2000	
destination_domain	VARCHAR(255)	NULLABLE	For per-destination throttling
concurrency_limit	INTEGER		

2.13 audit_logs

Column	Type	Constraints	Notes
id	BIGSERIAL	PRIMARY KEY	
actor_id	UUID	INDEX	
tenant_id	UUID	INDEX	
action	VARCHAR(128)	NOT NULL	
ip_address	INET	NOT NULL	
user_agent	TEXT		
metadata	JSONB		Extra context
timestamp	TIMESTAMPTZ	DEFAULT NOW()	

2.14 billing_subscriptions

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
tenant_id	UUID	FOREIGN KEY → tenants	
plan	VARCHAR(64)		
status	VARCHAR(32)		active / past_due / canceled
current_period_end	TIMESTAMPTZ		
payment_provider	VARCHAR(32)		Razorpay / Stripe
provider_subscription_id	VARCHAR(255)		

2.15 webhooks (outbound event notifications for enterprise clients)

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
tenant_id	UUID	FOREIGN KEY → tenants	
url	TEXT	NOT NULL	
event_types	TEXT[]		e.g. {mail.received, user.created}
secret	VARCHAR(255)		For HMAC signing
is_active	BOOLEAN	DEFAULT true	

2.16 deletion_requests (right-to-erasure compliance)

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
mailbox_id	UUID	FOREIGN KEY → mailboxes	
requested_by	UUID		
status	VARCHAR(32)	DEFAULT 'pending'	pending / processing / done
proof_hash	VARCHAR(255)		Cryptographic proof-of-deletion
completed_at	TIMESTAMPTZ		

Indexes not to overlook: mailboxes(domain_id, email), aliases(source), audit_logs(actor_id, timestamp), contacts_cache(mailbox_id, email), migration_jobs(status).

3. Phase-wise and Feature-wise Plan

Each phase is organized into four sections: **Infrastructure/Configuration**, **Backend API**, **Frontend**, and **Database Tables Involved**. Sub-features are broken down to the smallest practical unit.

Phase 1 — Foundation (Weeks 1-2)

Goal: Bring the mail engine online and allow basic login, send, and receive through the webmail client.

Infrastructure/Configuration

- Install Postfix; configure `virtual_mailbox_domains`, `virtual_mailbox_maps`, `virtual_alias_maps` via `proxy:pgsql`:
- Install Dovecot; configure `dovecot-sql.conf.ext` (`password_query`, `prefetch_userdb`)
- Configure `auth_username_format = %{user | username | lower}` for normalization
- Configure the LMTP socket: `mailbox_transport = lmtp:unix:private/dovecot-lmtp`
- Install Rspamd and ClamAV for basic spam scoring (DKIM signing is added in Phase 4)
- Decide mail storage format: `mdbox` vs. `Maildir`; directory pattern `/var/vmail/domains/%d/%n`
- Docker Compose file covering: Postfix, Dovecot, Rspamd, PostgreSQL, Redis, Node.js API, React webmail

Backend API (Node.js)

- Project scaffold (Express or Fastify), route/controller/service folder structure
- Database connection pool setup (choose ORM: `pg`, `Prisma`, or `Knex`)
- Migration tooling setup (`Prisma Migrate` or `node-pg-migrate`)
- `POST /auth/login` — authenticate via IMAP bind against Dovecot rather than a direct hash check
- Issue short-lived JWT access token and an HTTP-only refresh token cookie
- `POST /auth/refresh` — rotate the refresh token
- `POST /auth/logout` — revoke the refresh token
- Authentication middleware to verify the JWT on protected routes
- `GET /mail/folders` — list IMAP folders via `imapflow`
- `GET /mail/messages?folder=INBOX&page=` — paginated message list with envelope data
- `GET /mail/messages/:uid` — fetch and parse a full message (MIME)
- `POST /mail/send` — compose, build the MIME message, and send via `nodemailer/Postfix`
- `PATCH /mail/messages/:uid/flags` — toggle read/unread/flagged
- `DELETE /mail/messages/:uid` — move to Trash or expunge
- `POST /mail/messages/:uid/move` — move a message between folders
- Basic search endpoint (sequential IMAP SEARCH, prior to full-text indexing in Phase 2)
- Error-handling middleware and structured logging (`pino` or `winston`)

Frontend (React Webmail)

- Application shell and routing (login, inbox, compose, settings)
- Login screen (email and password form, error states)
- Inbox list view: sender, subject, snippet, date, read/unread styling
- Folder sidebar (Inbox, Sent, Drafts, Trash, Spam)
- Message detail view (HTML rendering, sanitized with `DOMPurify`)
- Compose window (To/Cc/Bcc, subject, body — plain textarea at this stage)
- Wire the send button to `POST /mail/send`
- Basic pagination or “load more” on the inbox

- Loading and empty states (empty inbox, no search results)

Database Tables Involved

tenants, domains, mailboxes, aliases, refresh_tokens

Phase 2 — Management (Weeks 3-4)

Goal: Allow an administrator to manage domains and users, import existing mailboxes, and enable fast search.

Infrastructure/Configuration

- Set up a Meilisearch or Elasticsearch container
- Configure the Dovecot FTS plugin (`fts_elastic` or `fts_flatcurve`) with `fts_autoindex`
- Set up Redis and BullMQ for background jobs
- Install the `imapsync` binary on worker nodes

Backend API

- `POST /admin/domains` — create a domain (tenant-scoped)
- `GET /admin/domains/:id/dns-check` — verify MX/SPF/DKIM/DMARC via DNS lookup
- `PATCH /admin/domains/:id` — activate or deactivate a domain
- `POST /admin/mailboxes` — create a user (including password hash generation)
- `GET /admin/mailboxes` — list mailboxes with pagination and search by email
- `PATCH /admin/mailboxes/:id` — update quota, name, or status
- `DELETE /admin/mailboxes/:id` — soft delete and trigger cleanup job
- `POST /admin/aliases` — CRUD for forwarders and catch-all addresses
- `POST /migration/jobs` — accept source IMAP credentials/OAuth2 tokens and push to BullMQ
- Migration worker: spawn `imapsync` as a child process and parse stdout progress
- WebSocket channel `migration:progress:{jobId}` — push percentage complete to the Admin UI
- Migration resume logic (idempotent, UID-based restart after failure)
- `GET /mail/search?q=` — proxy to Meilisearch/Elasticsearch instead of IMAP SEARCH
- `POST /sieve/rules` — convert visual-rule JSON into compiled Sieve script and push via ManageSieve
- `GET /sieve/rules` — fetch current rules to rebuild the UI
- ManageSieve client integration (upload, list, and activate scripts)

Frontend (React Admin Console)

- Admin login (separate, role-gated route)
- Domain onboarding wizard (Step 1: enter domain; Step 2: display DNS records to add; Step 3: verify)
- Domain list table with status badges (verified/pending/failed)
- User management table with create/edit/suspend/delete modals
- Alias management screen (forwarders, catch-all toggle)
- Migration wizard: source-provider selection → credentials form → progress bar (WebSocket-driven)
- Sieve visual rule builder (e.g. “If Subject contains X, move to Folder Y”)

- Wire the webmail search bar to the new full-text search endpoint (highlight matched terms)

Database Tables Involved

domains, dns_verification_logs, mailboxes, aliases, sieve_scripts, migration_jobs

Phase 3 — Collaboration: Calendar and Contacts (Weeks 5-7)

Goal: Integrate calendar and contacts with mail, and support meeting invitations.

Infrastructure/Configuration

- Install Radicale and configure the custom PostgreSQL storage backend
- Build the SSO bridge: validate mailbox JWTs against Radicale authentication

Backend API

- POST /calendar/events — create an event in Radicale via a CalDAV client
- GET /calendar/events?range= — fetch events for the calendar view
- PATCH /calendar/events/:id — update or reschedule an event
- DELETE /calendar/events/:id — cancel an event
- Generate iCalendar (.ics) files per RFC 5545 for outbound invitations
- Send invitations: attach the .ics file to an email via the Mail Service and Postfix
- Parse inbound .ics attachments — detect the text/calendar MIME part
- POST /calendar/rsvp — Accept/Decline/Tentative, updating Radicale and sending an RSVP email to the organizer
- Support recurring events (parse and generate RRULE)
- POST /contacts — CRUD for the personal address book (vCard 4.0 via Radicale CardDAV)
- GET /contacts/global — company-wide address book, managed by administrators
- Contact auto-suggestion: extract To/Cc/From headers on send/receive and update contacts_cache and Redis
- GET /contacts/suggest?q= — query the Redis cache with CardDAV as a fallback

Frontend (React Calendar App and Webmail Integration)

- Standalone calendar application shell sharing the SSO session with webmail
- Month, week, and day view components
- Event creation modal (title, time, attendees, location, recurrence)
- Inline “Accept/Decline/Tentative” widget on emails carrying an .ics attachment
- Contacts screen with personal and global tabs
- Recipient autocomplete in the compose window, wired to the contact-suggestion API
- Contact detail/edit modal

Database Tables Involved

caldav_carddav_objects, contacts_cache

Phase 4 — Security and Hardening (Weeks 8-9)

Goal: Deliver production-grade security and the polish features expected by enterprise clients.

Infrastructure/Configuration

- Configure Rspamd DKIM signing (per-domain key, automatic rotation)
- Configure SPF/DMARC evaluation rules in Rspamd (quarantine vs. reject policy)
- Integrate ClamAV with Rspamd for attachment scanning
- Configure Fail2ban rules for repeated failed IMAP/API login attempts
- Set up S3/MinIO buckets for large attachment offload

Backend API

- POST /auth/2fa/setup — generate a TOTP secret (speakeasy) and return a QR code
- POST /auth/2fa/verify — validate the TOTP before issuing the final JWT
- POST /auth/2fa/backup-codes — generate and hash backup codes
- POST /auth/2fa/disable — with re-authentication confirmation
- POST /admin/domains/:id/dkim/rotate — DKIM key generation and rotation
- Audit-log middleware that automatically records every administrative action (login, password reset, user deletion, etc.)
- GET /admin/audit-logs — filterable log-viewer API
- Route attachment uploads to S3/MinIO instead of local disk, using presigned URLs
- Undo Send: a delayed job queue that holds outbound mail for N seconds before handing it to Postfix
- POST /mail/schedule-send — queue a message for future delivery via BullMQ delayed jobs
- POST /mail/snooze — hide a message from the inbox until a specified time, resurfacing it via a scheduled job
- Email template CRUD (POST/GET/PATCH/DELETE /templates)
- POST /account/erasure-request — trigger the right-to-erasure workflow
- Erasure worker: purge the PostgreSQL row, issue an IMAP EXPUNGE, delete the corresponding full-text index entry, scrub S3, and write `deletion_requests.proof_hash`
- Rate limiting: postfix policy configuration synchronized with the `rate_limit_policies` table

Frontend

- 2FA setup screen (QR code scan and verification code entry)
- Backup codes display/download screen
- Login flow: add a TOTP entry step when 2FA is enabled
- Undo Send toast with a countdown and cancel button
- Schedule Send picker in the compose window
- Snooze option on the email list item (context menu)
- Template manager screen (save/insert signatures or canned replies)
- Admin: audit log viewer with filters (actor, action, date range)
- Admin: DKIM/DNS status dashboard per domain

Database Tables Involved

`two_factor_backup_codes`, `mailboxes` (2FA fields), `domains` (DKIM fields), `audit_logs`, `deletion_requests`, `rate_limit_policies`

Phase 5 — Scale and Mobile Readiness (Week 10 onward)

Goal: Multi-node scaling, mobile push notifications, billing, and high availability.

Infrastructure/Configuration

- Migrate the Docker Compose setup to Kubernetes manifests (or a Helm chart)
- Configure Dovecot `dsync` replication — enable notify and replication plugins in `10-mail.conf`
- Deploy two or more Dovecot nodes behind a TCP load balancer with TLS between nodes
- Configure PostgreSQL streaming replication (primary and standby)
- Set up a Firebase Cloud Messaging project for push notifications
- Implement storage-region selection per tenant for data-sovereignty compliance

Backend API

- Bridge IMAP IDLE events to FCM push notifications (in addition to the existing Web-Socket channel)
- `POST /devices/register` — register an FCM device token per mailbox
- Billing endpoints: `POST /billing/subscribe`, `POST /billing/webhook` (Razorpay/Stripe webhook handler)
- Plan-enforcement middleware (block actions when `max_users` is exceeded, plan has expired, etc.)
- Health-check endpoints for Kubernetes liveness/readiness probes
- Ensure session handling remains fully stateless across replicas (already true from Phase 1's JWT design)
- Failover test hooks to simulate node outage and verify load-balancer rerouting

Frontend

- Billing/subscription management screen (plan selection, invoice history)
- Mobile application (React Native), reusing the existing API and handling push notifications
- Admin: cluster health dashboard (node status, replication lag)

Database Tables Involved

`billing_subscriptions`, `webhooks`, and a new `devices` table for FCM tokens (to be added)

4. Cross-Cutting Concerns (Ongoing, Not Tied to a Single Phase)

- Input validation on every endpoint (zod or joi schemas)
 - Rate limiting on public-facing authentication endpoints for brute-force protection
 - API versioning strategy (/api/v1/...)
 - Automated testing: unit tests for services, integration tests against a test database, and end-to-end tests (Playwright/Cypress) for webmail flows
 - Error tracking (Sentry) wired into both the API and the React applications
 - Environment-based configuration (development/staging/production) — secrets are never hardcoded
 - API documentation (OpenAPI/Swagger, auto-generated)
 - GraphQL layer, if required alongside REST — scope should be decided explicitly rather than duplicating every endpoint
-

5. Suggested Sprint Grouping (Small Team)

Sprint	Maps To
Sprint 1-2	Phase 1 (Foundation)
Sprint 3-4	Phase 2 (Management)
Sprint 5-7	Phase 3 (Collaboration)
Sprint 8-9	Phase 4 (Security and Hardening)
Sprint 10+	Phase 5 (Scale and Mobile Readiness, ongoing)

Each sprint should close with a demo, a retrospective, and backlog grooming for the next phase's features.